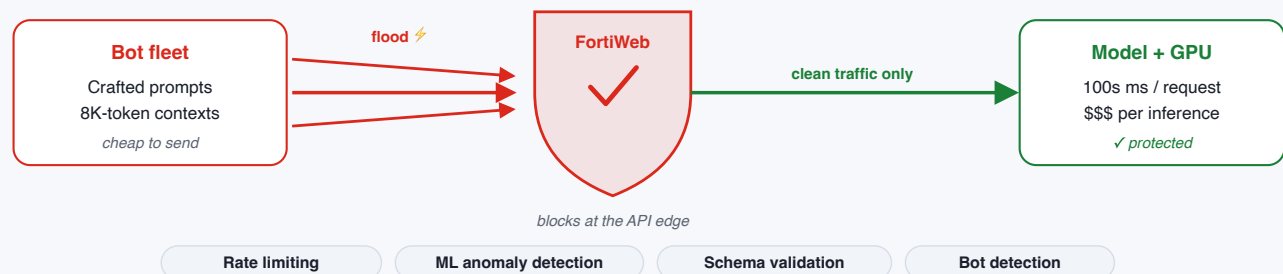




THREAT RESEARCH · AI APPLICATION PROTECTION

LLM models Application DDOS Attack

A single crafted prompt can burn hundreds of milliseconds of GPU time — so a *modest* flood becomes both an outage and a runaway cloud bill. FortiWeb shields the model at the API layer, dropping junk traffic before it reaches the backend:



Author: Sam V · Fortinet PreSales Cloud Published: June 2026 Reading time: ~10 min

Products: FortiWeb, FortiGuard Labs

Overview

Large language model services are uniquely fragile under coordinated load because **every request is computationally expensive**. A traditional web API call costs microseconds; an [LLM inference](#) request can take hundreds of milliseconds and consume significant [GPU](#)

[memory](#). That asymmetry — cheap to request, expensive to serve — is exactly what a [DDoS](#) actor exploits.

100s ms

GPU COST PER
INFERENCE

\$300+

COMPUTE BURNED IN A
10-MIN FLOOD

Token-aware

RATE LIMITING PER
AUTH TOKEN, NOT RAW
REQ/SEC

**API · Bot ·
Anomaly**

FORTIWEB COVERAGE
LAYERS

What are LLM DDoS attacks?

Unlike a classic volumetric flood, an LLM DDoS targets the *per-request cost* rather than the network pipe. A few hundred carefully shaped requests per second can degrade or topple a model endpoint that happily serves thousands of cheap requests. The attacker's economics are favorable: each malicious request is trivial to send and very expensive to serve.^{[1][3]}

Attack Vectors

- **[Token flooding](#)**: thousands of long-context prompts per second to exhaust GPU memory.
- **Expensive computation loops**: prompts that force costly model operations — e.g. forcing chain-of-thought reasoning on every request.
- **Concurrent session attacks**: bots open many connections and never close them, exhausting connection pools.

- **Prompt injection + volume:** prompts designed to generate maximally expensive responses, combined with request flooding.

THE ECONOMICS OF THE ATTACK

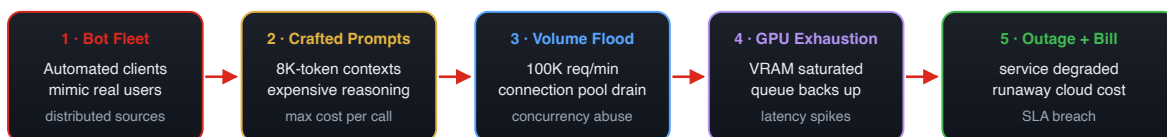
A chatbot using a frontier model costs roughly **\$0.03 per 1K tokens**. A 10-minute flood pushing 1M malicious tokens burns about **\$300 in compute alone** — before counting downtime, SLA penalties, and lost customer trust. The attacker spends almost nothing to inflict it.^[3]

Typical impacts include:

- Service slowdowns and increased latency (response-time degradation).
- API failures and complete outages during peak attacks.
- Thousands of dollars in wasted cloud compute (GPU hours consumed by junk traffic).
- Compliance violations (SLA breaches, regulatory uptime mandates).
- Reputational damage and lost customer confidence.

The attack chain at a glance

LLM DDOS — FROM CRAFTED PROMPT TO CLOUD BILL



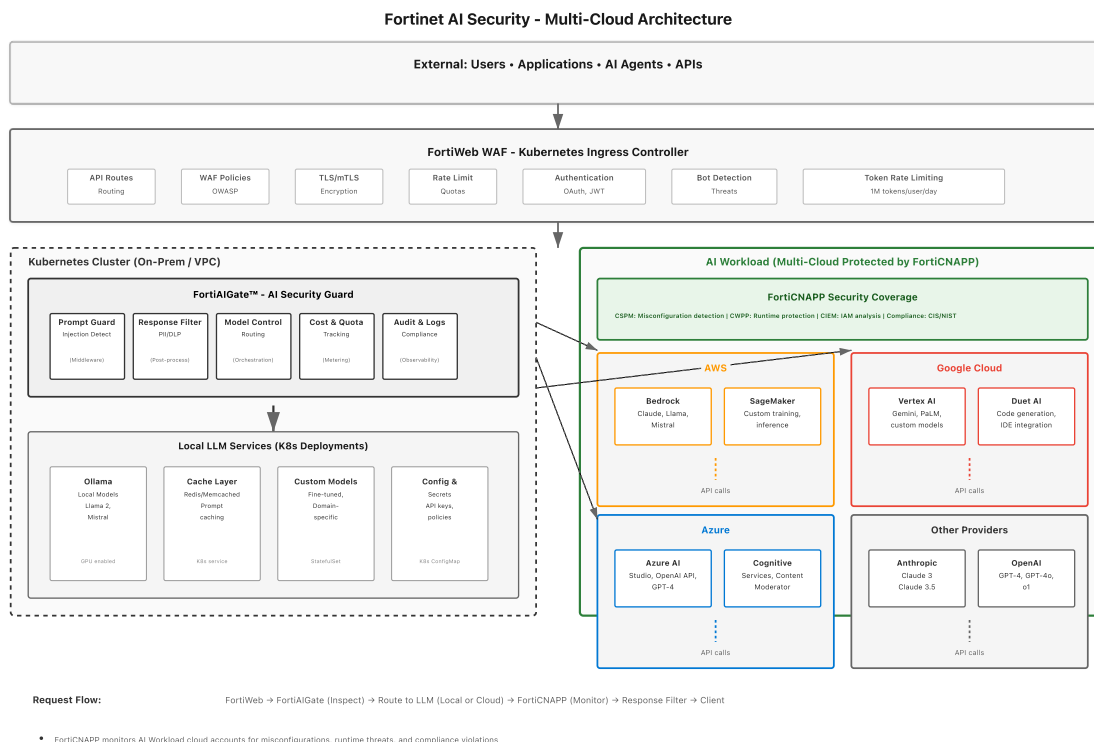
FortiWeb intercepts at stages 1–3 — before any traffic reaches the GPU.

FortiWeb stops the **structural and volumetric** dimension of an LLM DDoS at the API edge — request-size limits, request-rate limits, ML bot detection, OpenAPI schema validation, concurrency caps, and anomaly detection — and adds [MCP security](#) for agentic / tool-calling traffic. True **LLM-token cost metering and prompt-semantic** inspection is the role of [FortiAI Gate](#)^[9] in the Fortinet AI portfolio. Together they cover both layers; neither alone is "LLM-aware" on its own.^{[1][2]}

FortiWeb protection strategy

Best-Practice Architecture for Protecting AI Workloads

FortiWeb sits as a [reverse proxy](#) between users and your LLM service, inspecting every request and blocking attacks before they consume GPU resources:



FortiWeb Core protection mechanisms

FEATURE	HOW IT PROTECTS AGAINST LLM DDOS
API Protection	Secures LLM API endpoints by analyzing request content and enforcing WAF policies to block malicious traffic before it reaches the model.
Bot Mitigation	Detects and blocks automated bot traffic using ML-based fingerprinting and behavior analysis — no CAPTCHAs needed.
ML-Based Anomaly Detection	Learns a per-endpoint, per-parameter behavioral baseline and uses a two-layer ML model to flag deviations — request-rate spikes, abnormally large prompt fields, unexpected client behavior — with low false positives. ^[8]
Rate Limiting	Enforces granular quotas per API key, per user, and globally to prevent resource exhaustion.
API Discovery	Discovers LLM API endpoints and builds positive security models using OpenAPI / JSON schema validation.
Request Validation	Blocks malformed, oversized, or suspicious prompts before they reach the LLM backend.

FortiWeb Strategies

FortiWeb defends an LLM application DDoS in layers. The strongest place to start is *prevention* — stopping the automated traffic that powers a flood before it ever reaches a rate counter or the model.

PREVENTATIVE MEASURE

1

Bot & ML-based bot detection

An LLM application DDoS is, at scale, an **automated** attack — a fleet of clients hammering the API. So the highest-leverage preventative control is to separate **bot from human at the door** and drop the bots before any rate budget is spent. FortiWeb does this with three complementary engines:

- **Known-bot signatures & FortiGuard feeds** — block recognized bad bots, scrapers, and tools by signature and reputation.
- [ML-based bot detection](#) — a behavioral model scores each client human-vs-bot in real time, catching automation that mimics legitimate traffic and has no known signature.
- **Threshold & crawler heuristics** — rapid-request and crawl-pattern detection flags clients behaving like a flood source.

FORTIWEB · BOT MITIGATION (ILLUSTRATIVE)

```
# Web Protection > Bot Mitigation
set known-bots enable           # signature + FortiGuard bot reputation
set ml-based-detection enable   # behavioral model: human vs bot
set threshold-detection enable  # rapid-request / crawler heuristics
set action block                # ML-flagged bots dropped – no CAPTCHA
```

✓ Removes the **automated volume** that powers a flood before rate counters even engage, and stays **user-transparent** (no CAPTCHA). ✗ A determined attacker can rotate fingerprints — pair it with the ML anomaly engine below so rotated bots still surface as a traffic-shape anomaly.

2 WAF ML to detect suspicious activity

Bot detection separates bot from human; **ML-based anomaly detection** catches the attack even when individual clients look legitimate. FortiWeb's WAF ML learns a [behavioral baseline](#) for every endpoint and parameter, then flags the **volume, parameter, and behavioral** deviations a flood produces — with no signature or hand-written rule:

FORTIWEB · ML ANOMALY DETECTION (ILLUSTRATIVE)

```
# Web Protection > ML Based Anomaly Detection
set anomaly-detection enable    # learn per-URL / per-parameter baseline
```

```
set learning-mode auto          # build the behavioral model from live traffic
set action alert-then-block      # verify (2nd ML layer) before blocking
```

✓ Catches novel floods and rotated bots; the two-layer model keeps the false-positive rate low enough to leave in `block`. (Full treatment in *FortiWeb's real edge* below.)

COST-LIMITING STRATEGIES

Beyond prevention, these patterns address the *other* token — the LLM **text** token that drives GPU cost (not the auth token). Until native LLM-token metering ships, three practical patterns approximate per-prompt cost limiting on FortiWeb *today* — listed from roughest to most accurate. They are not mutually exclusive; the strongest posture combines all three. (CLI below is illustrative — the actual FortiWeb stanzas live under *HTTP Protocol Constraints / API Protection and Custom Rules*, not a single `rate-limit` block.)

1 Payload size as a proxy (rough)

Cap the request body size — payload bytes correlate loosely with token count. In FortiWeb this is an **HTTP protocol constraint / API-protection body-size limit**, not a rate-limit knob:

FORTIWEB · BODY-SIZE CONSTRAINT (ILLUSTRATIVE)

```
# Web Protection > Protocol Constraints (or API Protection body limit)
set max-body-length 32768      # ~32 KB ≈ ~8K tokens (GPT-3 tokenizer)
set action block
# Adjust the byte-to-token multiplier to match your model's tokenizer
```

✓ **Simple**, no external dependency. ✗ **Inaccurate** — compression, padding, and encoding differences skew the byte↔token ratio.

2 Header-based enforcement via the app's token decision (accurate)

FortiWeb can match a custom HTTP header in a Custom Rule and block on it — but it can't *sum* a numeric `X-Token-Count` into a per-minute budget; that aggregation isn't a FortiWeb primitive. So the app or API gateway owns the token math and keeps the running

total, then sets a **flag header** once a user exceeds budget. FortiWeb consumes the *decision*, not the count:

FORTIWEB · CUSTOM RULE (ILLUSTRATIVE)

```
# App/gateway counts tokens, sets a flag when the user is over budget:
#   X-Token-Budget-Exceeded: true

# FortiWeb Custom Rule – match the flag header, block:
match: HTTP header X-Token-Budget-Exceeded == "true"
action: deny (or redirect / challenge)
```

✓ **Accurate** — trusts the application's own token accounting. ✗ **Requires app-side instrumentation** to count tokens and set the flag; FortiWeb enforces the verdict, it doesn't meter tokens itself.

3 Endpoint-level rate limits (practical)

Different LLM operations have predictable cost profiles — limit each endpoint to match its weight, no token awareness needed:

FORTIWEB · PER-ENDPOINT LIMITS

```
/api/embeddings      -> 10 req/sec    # small payloads, fast
/api/completions      ->  2 req/sec    # large payloads, slow
/api/fine-tune        ->  1 req/hour   # resource-heavy
```

✓ **Practical**, no token awareness needed. ✗ **Doesn't account for prompt-size variance** within an endpoint.

COMBINE, DON'T CHOOSE

Endpoint limits (3) set sane per-operation ceilings, payload-size limits (1) catch oversized prompts cheaply, and a trusted `X-Token-Budget-Exceeded` flag (2) adds true token accuracy where you control the app. Layer all three under [ML-based anomaly detection](#) and the static thresholds stop being your only line of defense.

Per-auth-token rate limiting & credential-abuse defense

Continuing the strategies above: the cost-limiting patterns addressed the LLM **text** token; this section addresses the **auth token** — the credential carried in each request.

The single most useful thing FortiWeb can do in front of an external LLM API (Anthropic, OpenAI, Gemini, or your own) is to treat the **auth token / API key as the rate-limit key**. Instead of throttling by source IP — which collapses every user behind a NAT or a shared egress into one bucket — FortiWeb scopes the limit to each unique credential. That single design choice gives you per-key quotas, instant revocation, and blast-radius isolation when a key leaks.

Be precise about the boundary, though: FortiWeb rate-limits and blocks by the auth token's *value, pattern, and behavior* — it does **not** validate whether that token is legitimate. The authentication decision stays with the application or model provider. FortiWeb is the cheap, fast filter that strips malformed, brute-forced, and revoked credentials before they ever reach the backend.

1 Rate-limit per auth token

Extract the auth token from `Authorization: Bearer <token>` and scope the quota to each unique value — so one noisy or compromised key can't starve everyone else:

FORTIWEB · PER-TOKEN RATE LIMIT (ILLUSTRATIVE)

```
config waf http-request-rate-limit
  edit llm-per-token-quota
    set action block
    set uri-match-type prefix
    set uri "/v1/"                # OpenAI / Anthropic API paths
    set limit-to-request 1000     # per token, per period
    set request-period 3600       # 1 hour
    # Scope the limit to the credential, not the source IP
    set limit-by-key enable
    set key-type http-header
```

```

    set key-name "Authorization"
    set key-pattern "Bearer (.*)"      # capture the token value
  next
end

```

✓ Caps spend and abuse **per credential**. ✗ Only as granular as the keys you issue — give each app/team its own.

WORKED EXAMPLE: RATE-LIMITING CLAUDE CODE — MATCH THE *RIGHT* HEADER

A coding agent like **Claude Code** makes direct calls to the model API, so it carries a credential on every request to `/v1/messages` — a natural per-credential rate-limit target. But the header depends on how it authenticates, and this is where most rules go wrong:

- **API-key auth** (`ANTHROPIC_API_KEY=sk-ant-api03-...`) → the key travels in the `x-api-key` header. There is **no** `Authorization` header. [The Anthropic API uses](#) `x-api-key`, not bearer auth.
- **OAuth / subscription login** (`claude /login` or an OAuth proxy) → the token travels in `Authorization: Bearer <token>` (plus an `anthropic-beta: oauth-2025-04-20` header). OAuth tokens are a different format (e.g. `sk-ant-oat01-...`), not `sk-ant-api03-...`.

So `match "Authorization: Bearer sk-ant-..."` alone catches only the OAuth path. To cover both, scope the limit on whichever header is present:

FORTIWEB · PER-CREDENTIAL LIMIT, BOTH AUTH MODES (ILLUSTRATIVE)

```

config waf http-request-rate-limit
  edit claude-code-rate-limit
    set action block
    set limit-to-request 50
    set request-period 60
    set uri "/v1/messages"
    set limit-by-key enable
    # OAuth-auth Claude Code (subscription / proxy):
    set key-type http-header
    set key-name "Authorization"
    set key-pattern "Bearer (.*)"
  end
end

```

```

    next
end

# API-key-auth Claude Code puts the secret here instead – add a
# second rule keyed on x-api-key so it isn't silently missed:
#   set key-name "x-api-key"
#   set key-pattern "(sk-ant-.*)"

```

2 Enforce token format (regex)

Reject malformed or missing tokens up front. Anthropic keys begin with `sk-ant-`, OpenAI project keys with `sk-proj-` — FortiWeb can require that shape on the header before anything else runs:

FORTIWEB · TOKEN-FORMAT MATCH (ILLUSTRATIVE)

```

set header-match "Authorization: Bearer sk-ant-*" # require valid prefix
set action block                                # else drop at the edge

```

✓ Cheaply rejects junk. ✗ Confirms the token *looks* valid, not that it *is* valid.

3 Detect credential abuse & revoke fast

- Rate-limit sources sending repeated requests with **different or missing tokens** — the signature of [credential brute-forcing](#).
- **Block leaked or revoked keys instantly** with a Custom Rule that matches the token value — the moment a key shows up in a paste or alert, deny `sk-ant-<leaked>` at the edge without waiting on a backend redeploy.
- Track failed-auth patterns over time to feed rate-limit and anomaly thresholds.

CAPABILITY

OPERATIONAL BENEFIT

Per-token rate limits

One compromised key can't take down traffic for everyone else.

CAPABILITY	OPERATIONAL BENEFIT
Instant revocation	Block a leaked <code>sk-...</code> key at the edge immediately — no app deploy.
Per-key cost control	Cap LLM API spend per app or team, each on its own credential.
Usage visibility	See which keys are most active; spot a token that suddenly spikes.
Blast-radius isolation	Compromise is contained to a single token, not the whole service.

TOKEN-HANDLING HYGIENE

Never log full Bearer tokens — record only the last 4 characters or a hash, in FortiWeb logs and FortiAnalyzer alike. Rotate keys on a schedule, rate-limit aggressively on auth failures to blunt brute force, and when a key is confirmed compromised, revoke it through the provider's API *and* blocklist its shape on FortiWeb so the dead key never reaches them. FortiWeb screens the credential's value, shape, and behavior; the app still verifies its validity.

FortiWeb's real edge: ML-based anomaly detection

Here is the key point that rescues the WAF layer from the "no token semantics" limitation: **you don't need to understand a prompt to recognize that traffic stopped looking normal.** FortiWeb's [ML-based anomaly detection](#) learns a behavioral baseline for every API endpoint and parameter, then flags deviations from it — automatically, without signatures and without anyone writing a rule.^[8]

An LLM DDoS, no matter how it is crafted, has to change the *shape* of your traffic to do damage. That shape change is exactly what the model is built to catch:

- **Volume deviation:** a sudden request-rate spike against a learned baseline — the signature of a flood — registers as anomalous within minutes.
- **Parameter deviation:** abnormally long prompt fields or oversized payloads diverge from the learned normal value length and distribution for that endpoint, surfacing token-flooding behavior *structurally* even though the WAF never tokenizes the text.
- **Behavioral deviation:** unusual cadence, concurrency, or per-client patterns that don't match how real users have historically called the API.

WHY THE TWO-LAYER MODEL MATTERS HERE

FortiWeb's ML uses a **two-layer design**: the first layer builds the behavioral baseline and flags anomalies; the second layer validates whether a flagged anomaly is a genuine threat before acting. That second pass is what keeps the *false-positive rate* low — critical when the "block" decision could otherwise turn away paying customers during a traffic surge. Behavioral detection that you can't trust gets switched off; a two-layer model is what makes it safe to leave in **block** mode.^[8]

Paired with **ML-Based Bot Detection** (separating automated fleets from humans) and **ML-Based API Protection** (per-parameter abnormal-value analysis), this gives FortiWeb a genuine, shipping answer to LLM DDoS at the volumetric, structural, and behavioral layers — while [FortiAI Gate](#) owns the prompt-semantic layer above it.

Monitoring & detection

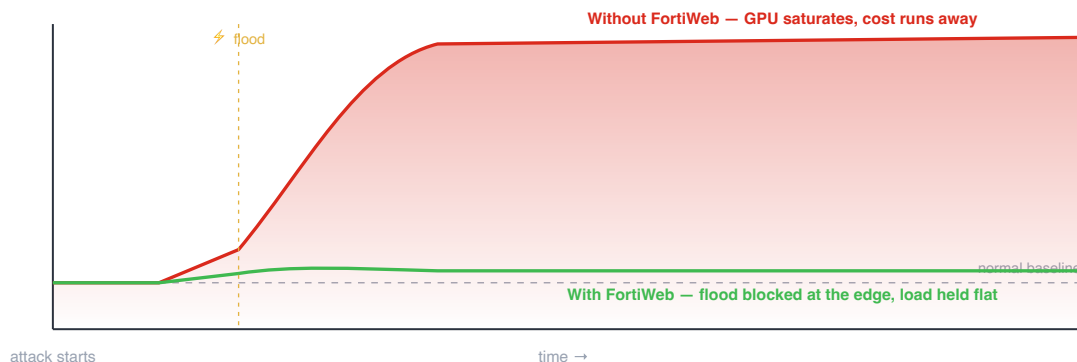
Key metrics to track

Each metric exists to answer one question: **is someone turning my GPU budget into their weapon, and is my mitigation costing real users anything?** The table below pairs every metric with *why* it matters and the signal that should make you act.

METRIC	WHY IT MATTERS	HEALTHY	ATTACK SIGNAL
Request volume	The first and bluntest indicator — a flood almost always shows up here before anywhere else. Without a baseline you can't tell a launch-day surge from an attack.	Flat or diurnal, near baseline	✗ > 200% of baseline in < 5 min
Average tokens/request	This is the cost dimension volume alone can't see. A small number of 8K-token prompts can burn more GPU than a huge number of short ones — token flooding hides under normal request counts.	Stable, matches typical prompt size	✗ Sharp climb toward max context
Bot vs. human ratio	DDoS at scale is automated. A rising bot share means a campaign is spinning up, even if total volume still looks normal.	Mostly human, known good bots	✗ Bot confidence > 90% at high rate
API error rates (4xx/5xx)	Tells you whether the backend is starting to buckle (5xx) or whether FortiWeb is actively shedding malicious load (4xx). The split tells you who's winning.	Low 4xx, near-zero 5xx	⚠ 5xx climbing = GPU saturating
Latency	The user-facing symptom. It proves the GPU queue is backing up — and proves inspection overhead stays negligible when traffic is clean.	< 100ms inspection overhead	✗ p95 spikes as queue grows
False-positive rate	Your mitigation's blast radius. A defense that blocks paying customers is its own outage — this is the metric that keeps thresholds honest.	Near zero; legit users unblocked	⚠ Rising = thresholds too tight

METRIC	WHY IT MATTERS	HEALTHY	ATTACK SIGNAL
Cost savings	Turns security into a dollar figure leadership understands: GPU spend you <i>avoided</i> by blocking junk traffic. This is the ROI line for the whole deployment.	Steady blocked-compute value	✓ Spikes during an attack = wins

WHAT THE METRICS LOOK LIKE DURING AN ATTACK — WITH VS. WITHOUT FORTIWEB



Recommended alerts

- Request-rate spike > 200% of baseline in < 5 minutes.
- Average tokens/request spike > 50% of baseline.
- Bot detection confidence > 90% for > 1M requests/hour.
- API error rate > 5% (potential attack or misconfiguration).
- Concurrent sessions > 95% of limit.

Defense in depth — layered view

No single control stops an LLM DDoS. Effective protection layers detection from the edge inward, so traffic is filtered, profiled, and rate-limited before a single token reaches the GPU.

Edge / API	API protection + schema validation	Rejects malformed and oversized prompts at the door.
Bot layer	ML fingerprinting + behavior analysis	Separates automated fleets from real users — no CAPTCHA.
Rate / token	Per-auth-token request budgets	Caps the load any one credential can inflict.
Anomaly	Dual-layer ML on volume and tokens	Catches novel patterns that fixed rules miss.
Backend	Protected LLM + GPU capacity	Only clean, budgeted traffic ever arrives.

Competitive comparison

CAPABILITY	FORTIWEB	AWS WAF + API GW	AZURE WAF + APIM
ML-based bot detection	✓ Native, no CAPTCHA	⚠ Limited (managed rules)	⚠ Limited (rules-based)
ML-based anomaly detection	✓ Two-layer behavioral baseline, low FP	✗ Static / managed rules	✗ Static / managed rules

CAPABILITY	FORTIWEB	AWS WAF + API GW	AZURE WAF + APIM
API schema validation	✓ OpenAPI / JSON schema	⚠ Basic in API Gateway	⚠ Basic in APIM
Cross-cloud consistency	✓ Same policies AWS/Azure/GCP/on-prem	✗ AWS-specific	✗ Azure-specific
Centralized management	✓ FortiManager	✗ Per-service console	✗ Per-service console
No CAPTCHA burden	✓ ML-based, user-transparent	⚠ CAPTCHA for bots	⚠ CAPTCHA for bots

PRESALES ANGLE

"FortiWeb brings ML-based bot and anomaly detection plus per-auth-token rate limiting across your multi-cloud infrastructure, while AWS and Azure WAF are single-cloud and lean on static rules."

Real-world risk scenarios

1 SaaS LLM chatbot (customer support)

Problem: a competitor attacks your support chatbot with 100K requests/min of expensive prompts. Cost impact: \$5K/hour.

FortiWeb solution:

- Per-auth-token rate limit: 100 req/min per credential (blocks the attacker's volume).
- Bot detection: ML flags automated traffic at 95%+ confidence.
- ML anomaly detection: the request-rate spike registers against baseline within minutes.

OUTCOME

Attack blocked in < 5 minutes; cost saved: \$5K/hour × 5 min ≈ **\$400** — and the service stays up for real customers.

2 Enterprise LLM RAG system (internal API)

Problem: a rogue employee writes a loop sending 10K max-length prompts. Resource exhaustion causes an outage.

FortiWeb solution:

- Request body-size limit: caps oversized prompts at the edge (≈8K-token payload ceiling).
- Per-auth-token request quota: throttles the looping credential before it exhausts the backend.
- ML anomaly detection: flags the unusual request pattern from the employee's client.
- Audit trail: FortiAnalyzer logs every request for forensics.

OUTCOME

Attack mitigated before service degradation; the audit trail provides evidence of the rogue behavior.

Fortinet AI security portfolio integration

For comprehensive LLM protection, combine FortiWeb with the broader Fortinet portfolio so infrastructure-layer and model-layer attacks are both covered:

COMPONENT	ROLE	THREAT COVERAGE
FortiWeb (WAF)	API / web-layer DDoS and bot mitigation	Request floods, resource exhaustion, bot & credential abuse
FortiAIGate (AI GW)	LLM-native runtime security between app and model	Prompt injection, jailbreaks, model poisoning, response/data leakage
FortiManager (Centralized mgmt)	Centralized policy management	Consistent enforcement across all LLM endpoints
FortiAnalyzer (Centralized logging)	Logging, analytics & forensics	Attack correlation, incident response, compliance audits
FortiAI Assist	GenAI security assistant for SecOps	Guided investigation, remediation, and automation across the portfolio

SEPARATION OF CONCERNS

FortiWeb handles infrastructure-layer DDoS and bots; [FortiAIGate](#) handles model-level attacks. Together they provide defense in depth.

Best practices & tuning

1. Baseline establishment (week 1–2)

- Enable anomaly detection in "monitor" mode (log-only, no blocking).
- Collect 7–14 days of traffic to establish baselines.
- Review bot detection confidence scores to tune sensitivity.
- Analyze legitimate user behavior to set appropriate rate limits.

2. Gradual enforcement (week 3–4)

- Enable rate limiting for the top 10% of heavy users (those near the limit).
- Switch anomaly detection to "block" mode with high thresholds.
- Monitor for false positives; whitelist legitimate traffic as needed.
- Adjust token limits based on user feedback.

3. Ongoing tuning

- Review weekly metrics: false-positive rate, attack volume, cost savings.
- Adjust bot detection sensitivity as attacker tactics evolve.
- Update API schemas when the LLM API changes.
- Collaborate with the security team to refine threat patterns.

4. Testing & validation

- Run an internal DDoS simulation to verify FortiWeb blocking.
- Test with legitimate heavy users to ensure no blocking.
- Validate that metrics are accurate before sharing with stakeholders.
- Document tuning decisions for future reference.

Why this matters

- LLM DDoS is a **cost attack**, not just an availability attack — defenders must budget against blast radius, not just downtime.^[3]
- Generic HTTP rate limits key on source IP and miss the credential; token-aware (per-auth-token) rate limiting is now a baseline requirement.^[1]
- Bot fleets increasingly mimic legitimate users, so ML detection beats static rules and avoids the CAPTCHA tax on real customers.^[2]

- Multi-cloud LLM estates need consistent policy — single-cloud WAFs fragment enforcement.^[2]

Summary & next steps

LLM DDoS attacks exploit expensive inference to cause outages and cost spikes. **FortiWeb protects** with API protection, ML-based bot detection, ML-based anomaly detection, and per-auth-token rate limiting — blocking attacks before they ever reach the GPU. To deploy: position FortiWeb as a reverse proxy, enable API schema validation, configure bot defense, scope rate limits per credential, and integrate with FortiAnalyzer for threat detection.

RECOMMENDED NEXT STEPS

1. **Architecture review:** map your LLM service topology and identify where FortiWeb sits.
2. **Baseline establishment:** deploy FortiWeb in monitor mode for 7–14 days.
3. **Policy configuration:** create API protection and rate-limiting policies from the baselines.
4. **Testing & validation:** run DDoS simulations to verify blocking without false positives.
5. **Production rollout:** deploy in active mode and monitor metrics.
6. **Continuous improvement:** tune policies against real-world attack patterns.

Sources — Fortinet & references

1. [Fortinet FortiWeb Documentation](#)

2. [FortiWeb Product Overview](#)
3. [OWASP API Security Top 10](#)
4. [OWASP Top 10 for LLM Applications](#)
5. [NIST Cybersecurity Framework](#)
6. [MITRE ATT&CK Framework](#)
7. Fortinet AI Security Portfolio Overview — contact your account team.
8. [FortiWeb 8.0.5 Administration Guide — ML-Based Anomaly Detection](#)
9. [Fortinet — FortiAIGate \(AI runtime security gateway for LLMs\)](#)